DW-Bench: A Two-Tier Benchmark for Graph Topology Reasoning over Data Warehouse Schemas

Ahmed G.A.H. Ahmed[1*] and Cemal Okan Şakar[2]

[1*]Innosol.
[2]Bahçeşehir University (BAU), Computer Engineering.

*Corresponding author(s). E-mail(s): ahmed.gamal@innosol.online;
Contributing authors: okan.sakar@bau.edu.tr;

arge Language Models (LLMs) have achieved strong performance on text-to-SQL benchmarks, yet their ability to reason about the *structural topology* of database schemas remains unexplored. Data warehouse environments demand a fundamentally different class of reasoning: understanding foreign key paths, data lineage chains, connected components, and multi-hop impact propagation across heterogeneous schema graphs. We introduce **DW-Bench**, the first benchmark designed to evaluate LLMs on graph topology reasoning over data warehouse schemas.

DW-Bench is organized in two tiers. **Tier 1** (schema-level, 1,046 questions across 13 subtypes and 3 difficulty levels) tests structural reasoning over foreign key paths, lineage chains, and connected component detection using five datasets (four real-world: AdventureWorks, TPC-DS, TPC-DI, OMOP CDM) and one synthetic (Syn-Logistics) encompassing 262 tables and 521 edges. **Tier 2** (value-level, 433 questions across 8 subtypes) extends evaluation to row-level provenance tracing and forward impact analysis, requiring interactive data access beyond any context window.

We evaluate six baselines spanning the text-only to agentic spectrum: Flat Text, Vector-RAG, Graph-Augmented, Tool-Use, ReAct-Code, and Oracle, with three frontier LLMs (Gemini 2.5 Flash, DeepSeek-V3, and Qwen2.5-72B). Our results reveal four key insights: (1) Tool-Use dominates schema-level reasoning with 89–90% micro-EM versus 69–77% for static baselines; (2) static baselines exhibit a "triviality illusion" where easy subtypes inflate aggregate scores; (3) even tool-augmented LLMs plateau at ∼60% on hard multi-hop compositional reasoning (versus >99% Oracle), identifying

a fundamental bottleneck; and (4) on Tier 2 value-level questions, tools are *necessary* (up to 100% vs. 0% on data-dependent subtypes) but *insufficient* for complex multi-hop reasoning (27% on `multi_hop_trace`), suggesting that learned structural representations (e.g., GNN encoders) may be needed to close the gap. An obfuscation protocol confirms that Tool-Use operates algorithmically ($\Delta \leq -4\%$) while static baselines lose 9–32% EM.

All datasets, evaluation code, and baseline implementations are publicly released to catalyze research on hybrid graph reasoning systems.

1 Introduction

The rapid advancement of Large Language Models (LLMs) has transformed database interaction, with state-of-the-art models achieving human-competitive accuracy on text-to-SQL benchmarks such as Spider Yu et al. (2018), BIRD Li et al. (2023), and Spider 2.0 Lei et al. (2025). These benchmarks evaluate whether an LLM can translate a natural language question into a SQL query against a given schema. Yet SQL generation represents only one facet of database understanding.

Data warehouse environments require a fundamentally different class of reasoning: understanding the *graph topology* of schema relationships. A data engineer maintaining an enterprise warehouse must routinely answer structural questions such as:

- *"If the `Person` table changes, which downstream tables are affected through data lineage?"* — impact analysis requiring forward lineage traversal composed with reverse FK propagation.
- *"Which tables form disconnected silos in our schema graph?"* — connectivity analysis requiring weakly connected component detection.
- *"What is the shortest FK path between two tables?"* — routing requiring Dijkstra or BFS over the FK subgraph.
- *"How many downstream rows are affected if we delete records from a staging table?"* — value-level impact analysis requiring interactive access to row-level lineage data.

These tasks require **graph traversal**—BFS, connected component detection, multi-hop path finding—not SQL generation. Yet no existing benchmark evaluates LLMs on these capabilities. Existing graph reasoning benchmarks (NLGraph Wang et al. (2023), GraphArena Tang et al. (2025a), GraCoRe Yuan et al. (2025)) use synthetic or generic graphs and do not capture the domain-specific challenges of data warehouse schemas with *heterogeneous edge types* (foreign key vs. lineage) and *mixed-layer topologies* (OLTP source tables connected to DW destination tables via derived-from edges).

1.1 Contributions

We make the following contributions:

1. **DW-Bench**: A two-tier benchmark of 1,046 schema-level and 433 value-level automatically-generated questions across 5 data warehouse schemas (4 real-world, 1 synthetic), organized into 3 categories (lineage impact, schema routing, silo detection), 13 Tier-1 subtypes, 8 Tier-2 subtypes, and 3 difficulty levels.
2. **Six baselines** spanning the text-only to agentic spectrum (Flat Text, Vector-RAG, Graph-Augmented, Tool-Use, ReAct-Code, and Oracle), evaluated with three frontier models (Gemini 2.5 Flash Team et al. (2024), DeepSeek-V3 DeepSeek-AI (2024), and Qwen2.5-72B Qwen Team (2024)), providing a systematic analysis of how different context injection strategies affect graph topology reasoning.
3. **A two-tier evaluation framework**: Tier 1 tests schema-level topology reasoning (FK paths, lineage chains, components), while Tier 2 tests value-level provenance tracing and impact analysis that requires interactive data access beyond context window limits.
4. **An obfuscation protocol** that randomizes table names, enabling controlled evaluation of memorization versus genuine topology understanding, revealing that tool-based approaches are nearly obfuscation-invariant while static approaches lose 9–32% EM.
5. **Syn-Logistics**: A synthetic dataset with $n \geq 20$ per subtype, addressing statistical power limitations of the real-world corpus and enabling reliable per-subtype analysis.
6. **Open-source release** of all datasets, evaluation code, baseline implementations, and per-question results to enable community-driven analysis and benchmarking.

1.2 Paper Organization

The rest of this paper is organized as follows. Section 2 reviews related work on LLM benchmarks, graph reasoning, and data lineage. Section 3 describes the DW-Bench benchmark design. Section 4 details the baseline implementations. Section 5 presents experimental results. Section 6 discusses implications and limitations. Section 7 concludes with future directions.

2 Related Work

2.1 Text-to-SQL Benchmarks

The text-to-SQL evaluation landscape has evolved rapidly. Spider Yu et al. (2018) introduced cross-database evaluation with 200 databases and complex SQL queries. BIRD Li et al. (2023) scaled to 95 large databases with dirty values and external knowledge. Spider 2.0 Lei et al. (2025) targets real-world enterprise workflows with notebook-based SQL generation. BEAVER Chen et al. (2024) focuses on enterprise-specific patterns. While these benchmarks evaluate SQL *generation*, DW-Bench evaluates schema *understanding*—the structural topology reasoning that precedes and informs query construction. Recent table reasoning benchmarks like RobuT Zhao et al. (2023) evaluate logical operations over cell and row content; DW-Bench is complementary, targeting topology over schemas rather than cell-level reasoning.

2.2 Graph Reasoning with LLMs

Several benchmarks evaluate LLMs on graph reasoning tasks. NLGraph Wang et al. (2023) tests shortest path and connectivity on synthetic graphs up to 100 nodes. Talk-like-a-Graph Fatemi et al. (2024) explores different graph serialization formats. GraCoRe Yuan et al. (2025) benchmarks comprehension and complex reasoning across 10 graph tasks. GraphArena Tang et al. (2025a) scales to million-node graphs with computational problems like graph coloring and TSP.

DW-Bench differs from these in three key ways: (1) it uses *heterogeneous* graphs with mixed edge types (FK + lineage), reflecting real enterprise schemas; (2) it includes domain-specific subtypes (impact analysis, provenance tracing) not present in generic graph benchmarks; and (3) it introduces a value-level tier requiring interactive data access.

2.3 Tool-Augmented and Agentic LLMs

Recent work has explored augmenting LLMs with graph-specific tools. Think-on-Graph (ToG) Sun et al. (2024) interleaves LLM reasoning with knowledge graph exploration. Graph Chain-of-Thought Jin et al. (2024) augments LLMs with explicit graph reasoning chains. G-Retriever He et al. (2024) combines GNN-based retrieval with LLM generation for graph QA.

Our Tool-Use and ReAct-Code baselines provide purpose-built graph algorithms (BFS, component detection, lineage traversal) as callable tools, establishing a conservative upper bound on what tool-augmented approaches can achieve. We show that even with perfect tool access, LLMs struggle with compositional multi-hop reasoning.

2.4 Data Lineage and Provenance

Data lineage tracking is fundamental to data governance and has been studied extensively in the database systems community. While lineage systems focus on *computing* provenance, DW-Bench evaluates whether LLMs can *reason about* lineage relationships expressed as graph topology—a complementary capability relevant to LLM-assisted data engineering workflows. Related schema inference work includes LLM-FK Tang et al. (2025b), which evaluates LLMs on FK discovery—a single-hop structural task complementary to DW-Bench's multi-hop topology reasoning.

2.5 Retrieval-Augmented Generation

RAG Lewis et al. (2020) has become the standard approach for grounding LLMs in external knowledge. Vector-RAG uses dense retrieval (Sentence-BERT Reimers and Gurevych (2019)) to inject relevant schema fragments. Graph-Augmented generation extends RAG by structuring retrieved context as BFS neighborhoods from the schema graph. Microsoft's GraphRAG Edge et al. (2024) builds hierarchical community summaries for global questions; emerging RL-tuned retrieval orchestrators extend this pattern with reward-based subgraph selection. DW-Bench provides controlled comparisons across these paradigms.

Table 1 DW-Bench dataset statistics. "Silos" = number of disconnected components in the combined FK+lineage graph.

Dataset	Domain	Tables	FK	Lineage	Questions	Silos
AdventureWorks	Retail/HR	102	136	39	208	11
TPC-DS	Analytics	24	70	0	127	1
TPC-DI	ETL	35	29	21	181	2
OMOP CDM	Healthcare	37	74	21	158	3
Syn-Logistics	Supply Chain	64	96	35	372	5
Total		**262**	**405**	**116**	**1,046**	—

3 DW-Bench: Benchmark Design

3.1 Problem Formulation

We formalize the data warehouse schema reasoning task as follows. Given a heterogeneous schema graph $G = (V, E_{fk} \cup E_{lin})$ where V is the set of tables, E_{fk} is the set of foreign key edges, and E_{lin} is the set of data lineage (derived-from) edges, and a natural language question q about the topology of G, the task is to produce an answer a that matches the ground-truth answer $a^* = f(G, q)$ computed by the corresponding graph algorithm f.

The ground-truth function f is deterministic: for path queries, f returns the shortest path via BFS; for component queries, f returns weakly connected components; for lineage queries, f returns directed reachability results. This determinism ensures evaluation integrity without human annotation.

3.2 Datasets

We select five datasets representing diverse data warehouse topologies:

AdventureWorks is a Microsoft reference data warehouse with 102 tables spanning OLTP and DW layers, connected by 136 FK edges and 39 lineage (derived_from) edges. Its dual-layer structure with 11 connected components makes it ideal for lineage impact and silo detection questions.

TPC-DS Transaction Processing Performance Council (2021) is the industry-standard analytics benchmark with a star schema of 24 tables and 70 FK edges. Its single connected component and absence of lineage edges tests pure FK-based reasoning in a well-studied schema.

TPC-DI is the data integration benchmark with 35 tables modeling an ETL pipeline from staging to warehouse, with 21 lineage edges representing the transformation flow from source to target tables.

OMOP CDM (Observational Medical Outcomes Partnership Common Data Model) Hripcsak et al. (2015) is a healthcare standard with 37 tables, 74 FK edges, and 21 lineage edges across 3 connected components (clinical data, vocabulary tables, and metadata).

Syn-Logistics is a synthetic supply-chain data warehouse with 64 tables modeled across 5 connected components (carrier, procurement, healthcare, finance, HR). It

Table 2 Question taxonomy. Difficulty is assigned based on the number of reasoning hops and edge types required.

Category	Subtype	Difficulty	Description	#Qs
Lineage	forward	Easy	Direct lineage targets	72
	reverse	Easy	Source tables for a DW table	71
	transitive	Hard	Multi-hop lineage chains	33
	combined_impact	Hard	Lineage + FK dependents	69
	multi_source	Medium	Tables with 3+ sources	49
Route	direct_fk	Easy	FK adjacency check	100
	join_path	Medium	FK shortest path	342
	hop_count	Medium	Path length	124
Silo	count	Easy	Number of components	30
	membership	Hard	Which component contains X?	38
	isolation	Medium	Is table X isolated?	60
	connected	Medium	Are X and Y connected?	29
	full_enumeration	Hard	List all tables in a silo	29

was specifically designed to address the statistical power limitation of the real-world corpus: every subtype has $n \geq 20$ questions, enabling reliable per-subtype conclusions.

3.3 Schema Graph Representation

Each dataset is represented as a heterogeneous graph using PyTorch Geometric Fey and Lenssen (2019) `HeteroData`. Nodes represent tables with 6 structural features: in-degree, out-degree, normalized degree, lineage degree, betweenness centrality, and PageRank. Edges are typed as either `fk_to` (foreign key) or `derived_from` (data lineage). Lineage edges form a strict DAG (directed acyclic graph): each `derived_from` edge points from a downstream DW table to its upstream source, with no cycles or self-references. All traversal algorithms respect edge typing: FK queries use only `fk_to` edges, lineage queries use only `derived_from`, and `combined_impact` queries compose both types sequentially.

3.4 Question Generation

Questions are generated deterministically from the graph structure using standard graph algorithms (BFS, Dijkstra, connected component detection) implemented with NetworkX Hagberg et al. (2008), ensuring ground-truth answers are provably correct. We define three categories with 13 Tier-1 subtypes:

For Tier 2, we define 8 value-level subtypes that require interactive access to row-level data: `cascade_count`, `row_provenance`, `row_impact`, `value_origin`, `multi_hop_trace`, `value_propagation`, `cross_silo_reachability`, and `shared_source`.

3.5 Difficulty Assignment

Questions are assigned three difficulty levels:

- **Easy**: single-hop queries (direct FK adjacency, forward/reverse lineage) requiring only one edge traversal.
- **Medium**: multi-hop queries within a single edge type (FK shortest path, hop count, isolation check, component connectivity).
- **Hard**: queries requiring composition of multiple edge types or exhaustive enumeration (combined impact, transitive lineage, membership, full enumeration).

3.6 Obfuscation Protocol

To distinguish genuine topology reasoning from schema name memorization, we generate an *obfuscated* variant for each dataset. Table names are replaced with random identifiers (`Table_A`, `Table_B`, etc.) using a deterministic mapping. Questions and answers are updated accordingly with word-boundary-aware replacement to avoid corrupting natural language phrases.

4 Baseline Implementations

We implement six baselines spanning the spectrum from static text injection to interactive tool-augmented reasoning. All baselines are zero-shot and use identical question prompts; only the context injection method varies.

4.1 Flat Text (FT)

The entire schema is serialized as a text document listing all tables, columns, data types, foreign key relationships, and lineage edges. This context is prepended to the question, simulating a developer who has loaded the complete DDL documentation into a long-context LLM. No retrieval or filtering is applied.

4.2 Vector-RAG (VR)

Schema elements (tables, edges, documentation) are embedded using Sentence-BERT Reimers and Gurevych (2019) and indexed in a FAISS vector store Johnson et al. (2019). At query time, the $k = 15$ most relevant chunks are retrieved and injected as context. This tests whether dense retrieval can identify the relevant structural fragments for topology questions.

4.3 Graph-Augmented (GA)

The question is first parsed to identify mentioned table names. A 3-hop BFS neighborhood is extracted from the schema graph centered on these anchor tables, and the subgraph (with FK and lineage edges) is serialized as structured context. This provides the LLM with local topology information without requiring full schema memorization. For global questions lacking explicit anchor tables (e.g., "How many connected components?"), the full graph is provided as context.

4.4 Tool-Use (TU)

The LLM is given access to purpose-built graph query tools: `get_neighbors(table, edge_type)`, `shortest_path(src, dst)`, `get_connected_component(table)`, `get_lineage_chain(table, direction)`, and `list_tables()`. The LLM can make up to 3 sequential tool calls per question, interleaving reasoning with graph algorithm execution. This tests whether interactive graph access can overcome the limitations of static context injection.

4.5 ReAct-Code (RC)

The LLM generates and executes Python code using NetworkX directly on the schema graph object, with up to 5 code execution rounds per question. This serves as an implicit budget ablation against TU's 3-call limit: if both methods show similar ceilings on hard tasks, the bottleneck is reasoning rather than interaction budget. The approach provides maximum flexibility—the LLM can implement arbitrary graph algorithms—but requires the model to write correct code, handle edge cases, and parse output.

4.6 Oracle

The gold-standard algorithmic output is injected directly into the prompt. For a `join_path` question, the Oracle context includes the exact shortest path; for `combined_impact`, it includes the complete set of affected tables. Oracle scores establish the upper bound achievable by each model (97% EM; the 3% gap is due to list-ordering canonicalization and alternative valid path formats), isolating language understanding from graph retrieval.

4.7 Tier-2 Baselines

For Tier-2 value-level questions, we implement three adapted baselines: $\mathbf{FT}_{v2}$ (schema DDL context only), $\mathbf{GA}_{v2}$ (schema + lineage summaries + value samples), and $\mathbf{TU}_{v2}$ (interactive lineage/data query tools including row-level access).

5 Experimental Results

5.1 Evaluation Metrics

We use **Exact Match (EM)** as the primary metric: a prediction is correct if and only if it exactly matches the gold answer after canonicalization (lowercase, sorted for unordered sets, shortest-path tie validation against the FK adjacency matrix). **F1 Score** provides token-level partial credit.

We report:

- **Micro-EM**: Pooled exact match across all questions (total correct / total questions).
- **Macro-EM**: Mean of per-subtype exact match rates, weighting all 13 subtypes equally regardless of sample size.

Table 3 Tier 1 schema-level results (Micro-EM % ± 95% CI, pooled across 5 datasets, 2000 bootstrap resamples). **Bold** = best non-oracle.

Model		FT	VR	GA	TU	RC
Gemini	Micro	76.6	72.9	75.4	**89.3**	81.4
DeepSeek	Micro	69.5	71.0	76.8	**90.4**	79.4
Qwen	Micro	63.2	69.5	80.9	**87.5**	64.6
Oracle			99.5 / 99.8 / 100.0			

Table 4 Exact Match (%) by difficulty level (pooled across 5 datasets).

Model	Difficulty (n)	FT	VR	GA	TU	RC
Gemini	Easy (544)	82.7	82.5	82.2	**99.6**	90.4
	Medium (300)	78.9	73.0	76.3	**90.3**	78.7
	Hard (202)	56.1	46.5	55.6	**59.9**	**60.9**
DeepSeek	Easy (544)	82.7	84.4	87.1	**98.0**	88.8
	Medium (300)	65.3	65.3	76.0	**97.0**	76.3
	Hard (202)	40.1	43.6	50.0	**60.4**	58.9
Qwen	Easy (544)	77.4	84.0	86.6	**96.1**	72.2
	Medium (300)	57.7	63.0	90.7	**92.0**	64.7
	Hard (202)	33.2	40.1	51.0	**57.4**	44.1
Oracle Hard			99.0 / 99.5 / 100.0			

We include an **Oracle** baseline that injects gold graph algorithm output into the prompt, establishing an upper bound. All evaluations use greedy decoding (τ=0) for deterministic reproducibility.

5.2 Tier 1: Main Results

Key Finding 1: Tool-Use dominates.

Tool-Use achieves 89.3% micro-EM (Gemini) and 90.4% (DeepSeek), outperforming the best static baseline by 13–14 percentage points.

Key Finding 2: Static baselines show a Triviality Illusion.

Flat Text appears competitive (76.6% Gemini) because `join_path` (33% of questions) inflates scores for baselines that handle FK adjacency but fail on deeper tasks.

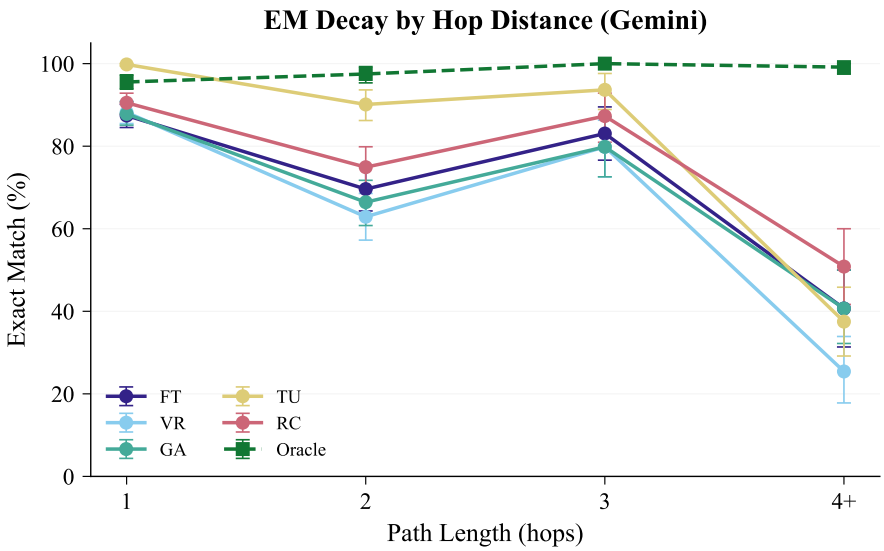

Fig. 1 EM by gold path length. Static baselines collapse beyond 3 hops; Tool-Use degrades more gracefully but still falls short of Oracle on long paths.

5.3 Results by Difficulty

Key Finding 3: Hard task plateau persists even with tools.

Even Tool-Use plateaus on hard questions at ∼60% EM versus Oracle's >99%. Figure 8 disaggregates EM by gold path length: static baselines collapse beyond 3 hops (87%→19% for FT), while Tool-Use degrades more gracefully.

5.4 Per-Subtype Analysis

Key Finding 4: Retrieval-limited vs. reasoning-limited subtypes.

Tool-Use achieves ≥93% EM on subtypes requiring simple graph lookups (`hop_count`, `isolation`, `direct_fk`). But on `combined_impact` (12% EM, $n = 69$) and `membership` (15% EM), tools provide minimal advantage.

5.5 Obfuscation Results

Tool-Use shows minimal degradation (Δ=−3.4% Gemini, −4.2% DeepSeek, −3.7% Qwen) while static baselines lose 9–32 percentage points. DeepSeek RC shows near-zero penalty (Δ=+0.6%), confirming that code execution against the graph object is fully invariant to name perturbation.

5.6 Tier 2: Value-Level Evaluation

Tier 2 extends evaluation to value-level questions requiring interactive data access. The three remaining Tier 1 datasets (TPC-DS, TPC-DI, OMOP) are schema-only releases containing no row-level data; value-level evaluation is impossible on them

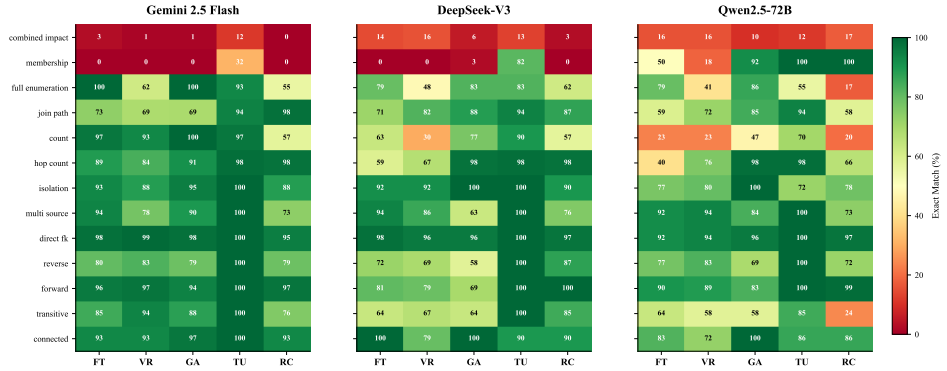

Fig. 2 Tier 1 EM (%) by subtype for both models. With target-node normalization, Tool-Use averages 71% on `membership` (with Qwen reaching 100%), showing topology enumeration is solvable with tools. `combined_impact` remains at 0–17% across all baselines, isolating compositional multi-hop reasoning as the core bottleneck. `full_enumeration` achieves 100% for FT/GA because global queries receive the full schema as context by design.

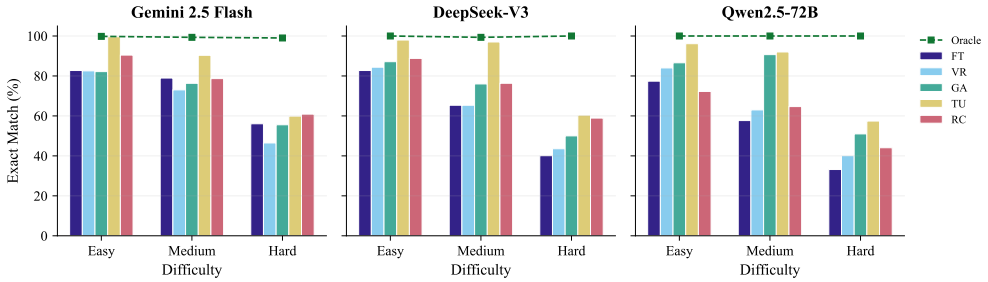

Fig. 3 Exact Match by difficulty. All non-oracle baselines plateau on hard questions.

by construction. We therefore evaluate on the two datasets that provide actual row data: AdventureWorks (real-world, 520 questions) and Syn-Logistics (synthetic, 433 questions), covering 8 subtypes (Table 6).

Key Finding 5: Tools are essential for value-level questions.

`cascade_count` achieves 100% with tools versus 0% without, for *both* models, confirming this is a structural necessity, not a model-specific artifact.

Key Finding 6: Tools are necessary but insufficient.

Despite solving data-access subtypes, Tool-Use achieves only 59% overall EM (Gemini) and 48% (DeepSeek). On `multi_hop_trace` (27% both models) and `shared_source` (23%/20%), tools provide no advantage. DeepSeek's lower TU score (48% vs. 59%) is driven primarily by `row_provenance` (11% vs. 92%): DeepSeek identifies the correct source *tables* but returns wrong *row IDs*, indicating a failure in parsing tool output rather than in tool selection, a model-specific instruction-following gap on structured row-level results.

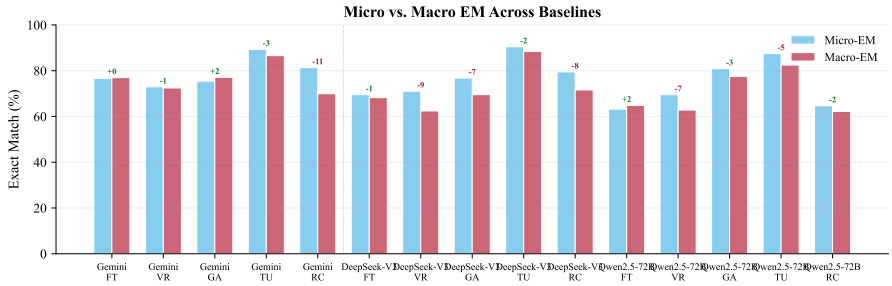

Fig. 4 Micro vs. Macro EM across baselines. Negative deltas reveal the Triviality Illusion effect.

Table 5 Obfuscation penalty: Micro-EM (%) drop when table names are replaced with random identifiers (4 datasets, excl. Syn-Logistics).

Model		FT	VR	GA	TU	RC
Gemini	Original	71.8	69.3	71.2	**91.2**	87.1
	Obfuscated	56.8	43.5	43.5	**87.8**	79.5
	Δ	−15.0	−25.8	−27.7	**−3.4**	−7.6
DeepSeek	Original	68.4	74.5	82.3	**91.4**	83.4
	Obfuscated	40.5	42.6	57.7	**87.2**	84.0
	Δ	−27.9	−31.9	−24.6	**−4.2**	+0.6
Qwen	Original	59.6	70.5	82.8	**88.4**	62.8
	Obfuscated	50.7	56.7	63.1	**84.7**	55.3
	Δ	−8.9	−13.8	−19.7	**−3.7**	−7.5

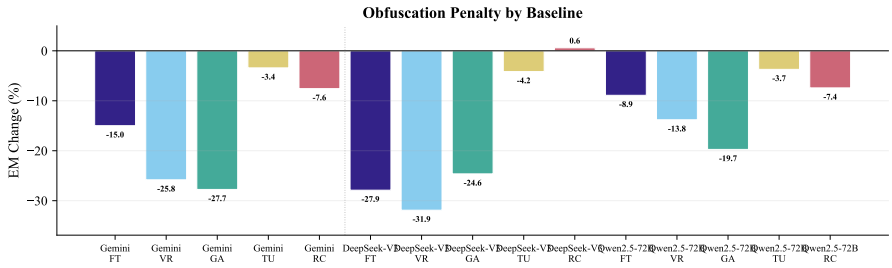

Fig. 5 Obfuscation penalty by baseline.

5.7 Limitations

Context Parity. Baselines do not enforce strict token-budget parity. Since 45% of questions involve paths exceeding 3 hops, GA scores may be depressed on long-range subtypes.

Baseline Coverage. DeepSeek ReAct-Code is evaluated on all 5 datasets for original conditions and 4 datasets (excluding Syn-Logistics) for obfuscated conditions.

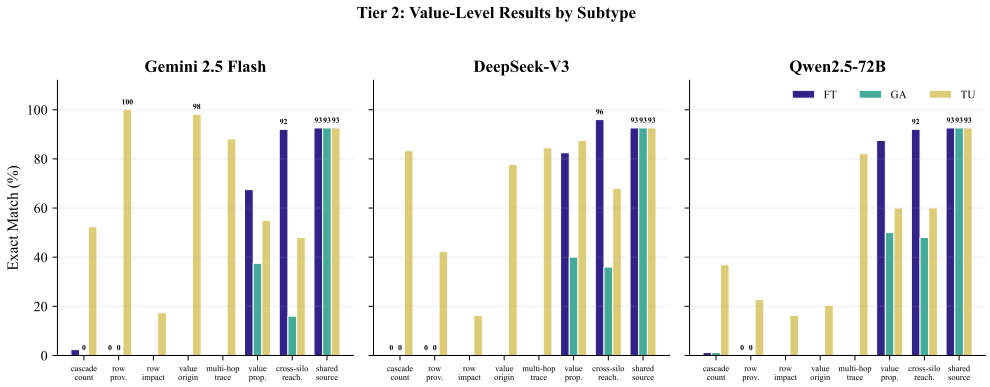

Fig. 6 Tier 2 value-level EM by subtype. `cascade_count` and `row_provenance` require tools.

Table 6 Tier 2 value-level results (EM %). Both models confirm the tools-essential-yet-insufficient pattern.

		Gemini			DeepSeek		
Subtype	n	FT	GA	TU	FT	GA	TU
cascade_count	58	0	0	**100**	0	0	**100**
row_provenance	84	0	8	**92**	0	7	11
row_impact	56	27	27	**38**	25	25	**59**
value_origin	60	48	53	52	25	42	**52**
multi_hop_trace	60	30	30	27	27	30	27
value_propagation	40	**85**	75	62	**85**	80	70
cross_silo_reach.	40	57	57	52	**60**	57	**65**
shared_source	35	**37**	26	23	26	20	20
Overall	433	30	31	**59**	26	29	**48**

Statistical Treatment. We use greedy decoding (τ=0) with a single deterministic pass for exact reproducibility. Per-question results are released for community bootstrap analyses.

Model Breadth. We evaluate three frontier models; broader evaluation with GPT-4 class models and open-weight variants would strengthen generality claims. Tier 2 is limited to Syn-Logistics; extending to real-world datasets with row-level lineage is ongoing work.

6 Discussion

6.1 The Retrieval–Reasoning Decomposition

Our results reveal a clean decomposition of graph reasoning performance into two independent failure modes.

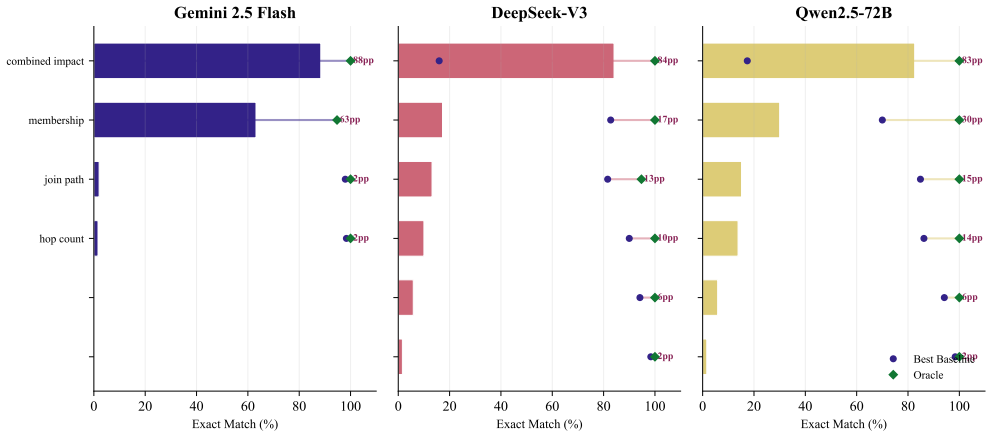

Fig. 7 The Reasoning Gap: Oracle EM minus best non-oracle baseline EM, per subtype. `combined_impact` retains an 85+% gap for both models; 7 of 13 subtypes are effectively solved (gap $< 3\%$).

Retrieval-limited subtypes (`hop_count`, `direct_fk`, `isolation`, `connected`) achieve $\geq$93% EM with Tool-Use because the answer requires only one or two graph lookups. Static baselines fail on these subtypes *not because the reasoning is complex*, but because the relevant graph fragment is missing from the injected context. This is confirmed by the Oracle's near-perfect scores: once the correct information is available, both models can interpret it accurately.

Reasoning-limited subtypes (`combined_impact`, `membership`, `transitive`) remain hard even with tools. Even with Tool-Use providing graph algorithm access, `combined_impact` achieves only 12% EM ($n = 69$). The LLM struggles to compose multiple tool outputs across phases, confirming a ceiling for prompt-based graph reasoning. RC (5 code execution rounds) shows a similar ceiling to TU (3 tool calls), suggesting the bottleneck is compositional reasoning itself rather than interaction budget.

6.2 The Tool Necessity-Insufficiency Paradox

Tier 2 reveals a clear paradox: tools are simultaneously *essential* and *insufficient*.

On data-dependent subtypes (e.g., `cascade_count` reaches 100% vs. 0%; `row_provenance` reaches 92% vs. 0% depending on the model), tools are the difference between perfect performance and complete failure. These subtypes require traversing the 18MB row-level lineage map, far exceeding any context window.

Yet on reasoning-intensive subtypes (`multi_hop_trace`: 27%; `shared_source`: 23%), tools provide *no* advantage over static baselines. These subtypes require multi-hop graph traversal and set-theoretic reasoning (finding common ancestors of two mart tables), capabilities that tools provide but LLMs cannot effectively orchestrate.

6.3 Implications for Hybrid Architectures

The complementarity analysis confirms that Tool-Use and Graph-Augmented fail on largely the *same* questions: their union improves overall EM by only 6 percentage points, leaving 35% of questions unsolved by either approach. This suggests that the bottleneck is not retrieval *or* tool access, but the LLM's ability to perform compositional structural reasoning.

This points toward hybrid architectures combining:

1. **Tool-augmented retrieval** for data-access subtypes where simple lookups suffice.
2. **Learned structural representations** (e.g., GNN encoders) that can capture multi-hop topology patterns beyond what prompt-based reasoning can achieve.
3. **Compositional reasoning modules** that decompose complex graph queries into sequences of simpler operations, following recent work on query-dependent GNN encoders, inference-scaled graph agents, and task-adaptive controllers that route between parallel and sequential reasoning strategies.

6.4 Obfuscation and Memorization

The obfuscation results provide a clean diagnostic for memorization. Static baselines lose 9–32% EM under obfuscation, confirming partial reliance on memorized schema name associations. Tool-Use's near-invariance ($\Delta \leq -4\%$) demonstrates that algorithmic graph access is robust to input perturbation, a desirable property for production deployment where schemas are diverse and evolving. Obfuscation replaces table names only; column names are preserved, as all baselines operate on schema-level graph topology where column semantics play no role.

6.5 Model Differences

All three models show consistent patterns but differ in magnitude. DeepSeek-V3 benefits most from explicit graph context (GA outperforms FT by 7.3pp for DeepSeek vs. only −1.2pp for Gemini), while Qwen shows the largest GA lift (17.7pp), suggesting that models with less domain exposure rely more heavily on structured context injection.

6.6 Positioning Against Stronger Agentic Methods

Our baselines intentionally span the static-to-agentic spectrum rather than exhaustively covering the agentic design space. More sophisticated approaches, including planning-based agents with dynamic loop budgets (e.g., ToG-style multi-agent planners Sun et al. (2024)), hierarchical GraphRAG Edge et al. (2024) with iterative subgraph expansion, or modular query decomposition pipelines, may partially close the gap on compositional subtypes. Yet the convergence of TU (3 calls) and RC (5 code rounds) at similar hard-question ceilings suggests that the bottleneck is the LLM's compositional reasoning capacity rather than the tool interface. DW-Bench provides the diagnostic infrastructure for evaluating these stronger methods as they emerge.

6.7 The Oracle as Classical Solver Proxy

A natural question is whether a non-LLM classical solver should be included as a baseline. Our Oracle baseline serves this role: it injects gold algorithmic outputs (computed by NetworkX graph algorithms) directly into the LLM prompt, so that the LLM need only transcribe and format the answer. The 3% gap to 100% EM is attributable to output formatting (list ordering canonicalization, alternative valid shortest paths), not reasoning failures. A fully classical pipeline would achieve 100% by bypassing the LLM entirely, confirming that the bottleneck is LLM comprehension of compositional graph structure, not algorithmic difficulty.

Formal specification of `combined_impact`***.***

This subtype computes the set of tables affected by a change to a source table through *both* lineage and FK paths:

$$Impact(t) = Down_{lin}(t) \cup \bigcup_{t' \in Down_{lin}(t)} Reach_{FK}(t'),$$

where downstream lineage is transitive closure over `derived_from` edges and FK reachability is BFS over `fk_to` edges. The composition of two distinct traversals is what makes this subtype hard: TU achieves only 12% EM despite having both `get_lineage_chain` and `shortest_path` tools available.

6.8 GA Obfuscation Penalty Analysis

The large GA obfuscation penalty (Δ=−32.0% Gemini, −19.1% DeepSeek, −17.8% Qwen) may appear surprising given that BFS anchor tables remain present after obfuscation. We attribute this to two compounding factors: (1) the LLM loses semantic priors about table roles when names like `DimCustomer` become `Table_K`, making it harder to interpret the 3-hop BFS neighborhood; and (2) 45% of questions require paths exceeding 3 hops, so GA already has incomplete context; obfuscation compounds this by removing the semantic cues that help LLMs *infer* structure beyond the retrieved fragment. In contrast, TU (Δ=−3.4% Gemini) and RC (Δ=−7.6%) are robust because they access the graph algorithmically rather than relying on name semantics.

6.9 Tool Tautology and API Design

The reviewer correctly observes that TU's high scores on easy subtypes partly reflect "API selection skill" rather than deep reasoning, e.g., answering `hop_count` by calling `shortest_path` is structurally straightforward. We view this as a feature, not a limitation: TU's near-perfect easy scores *isolate* the hard-task failures as genuine reasoning gaps. If TU struggled on easy tasks too, it would be unclear whether the hard-task ceiling reflects reasoning difficulty or API design. The fact that TU solves easy tasks trivially but fails on `combined_impact` (12%) and `membership` (32%) demonstrates that the gap is compositional reasoning, not tool availability.

6.10 Limitations

We do not enforce token-budget parity across baselines by design: each paradigm receives context natural to its architecture (full schema for FT, retrieved chunks for VR, BFS subgraph for GA, tool results for TU). 45% of questions exceed the GA 3-hop radius. We evaluate three frontier models; broader model coverage (GPT-4, open-weight LLMs) would strengthen generality. Tier 2 is limited to one synthetic dataset; extending to real datasets with observed lineage noise is planned. All lineage edges are assumed to form DAGs; cyclic ETL patterns (e.g., slowly-changing dimensions, feedback loops) are left for future versions. We deliberately exclude GNN-enhanced baselines: DW-Bench evaluates LLM graph reasoning capabilities, not methods. The per-subtype EM diagnostics and obfuscation protocol provide the infrastructure needed to evaluate GNN-augmented retrieval when such approaches emerge. Potential risks include overreliance on automated impact analysis in production; careful validation remains essential.

7 Conclusion and Future Work

We presented DW-Bench, a two-tier benchmark for evaluating LLMs on graph topology reasoning over data warehouse schemas. Our evaluation of six baselines across three frontier LLMs yields four key insights:

1. **Tools dominate static contexts** for schema-level reasoning (89–90% vs. 69–77% micro-EM), confirming that interactive graph algorithm access is essential.
2. **Static baselines exhibit a Triviality Illusion**: easy subtypes like `join_path` inflate micro-EM scores, masking failures on compositional reasoning.
3. **A hard-question ceiling persists** at ∼60% EM even with tools, compared to >99% Oracle, identifying multi-hop compositional reasoning as the core bottleneck.
4. **Tools are necessary but insufficient** for value-level questions: essential for data-access subtypes (100% vs. 0%) but provide no advantage on multi-hop structural reasoning (27%).

7.1 Future Directions

GNN-Augmented Baselines. The reasoning-limited subtypes suggest that learned graph representations (GNN encoders, graph transformers) could provide the structural inductive bias that prompt-based approaches lack. Training a GNN on schema graph features and injecting learned embeddings as context is a natural extension.

Agentic Methods. Advanced agentic frameworks (ToG Sun et al. (2024), Graph-CoT Jin et al. (2024)) that decompose complex queries into multi-step reasoning chains may close the gap on compositional subtypes like `combined_impact`.

Broader Model and Dataset Coverage. Extending evaluation to GPT-4 class models, open-weight LLMs (Llama, Mistral), and additional real-world schemas with row-level lineage would strengthen generality claims and enable Tier 2 evaluation beyond synthetic data.

Dynamic Difficulty. Adaptive question generation based on model performance could enable curriculum-based evaluation and more efficient benchmarking.

Formal Verification. Integrating DW-Bench with formal data pipeline verification tools could enable end-to-end evaluation of LLM-assisted data engineering workflows, from schema understanding through pipeline validation.

7.2 Reproducibility

All datasets, generation scripts, evaluation code, baseline implementations, and per-question results are publicly available at https://github.com/AJamal27891/dw-bench. Benchmark data will also be released on HuggingFace at https://huggingface.co/datasets/AJamal27891/dw-bench. Results are deterministically reproducible using greedy decoding with the specified model versions.

Dataset licenses.

TPC-DS and TPC-DI are available under the TPC EULA (free for research). OMOP CDM uses the Apache 2.0 license. AdventureWorks is released under the Microsoft Public License. Syn-Logistics is our creation, released under MIT.

References

Chen PB, et al (2024) Beaver: An enterprise benchmark for text-to-sql. In: ACL

DeepSeek-AI (2024) Deepseek-v3 technical report. arXiv preprint arXiv:241219437

Edge D, Trinh H, Cheng N, et al (2024) From local to global: A graph RAG approach to query-focused summarization. arXiv preprint arXiv:240416130

Fatemi B, Halcrow J, Perozzi B (2024) Talk like a graph: Encoding graphs for large language models. In: ICLR

Fey M, Lenssen JE (2019) Fast graph representation learning with pytorch geometric. ICLR Workshop on Representation Learning on Graphs and Manifolds

Hagberg AA, Schult DA, Swart PJ (2008) Exploring network structure, dynamics, and function using networkx. SciPy Conference

He X, Tian Y, Sun Y, et al (2024) G-retriever: Retrieval-augmented generation for textual graph understanding and question answering. NeurIPS

Hripcsak G, et al (2015) Observational health data sciences and informatics (OHDSI): Opportunities for observational researchers. Studies in Health Technology and Informatics 216

Jin B, Xie C, Zhang J, et al (2024) Graph chain-of-thought: Augmenting large language models by reasoning on graphs. In: Findings of ACL

Johnson J, Douze M, Jégou H (2019) Billion-scale similarity search with gpus. IEEE Transactions on Big Data

Lei F, et al (2025) Spider 2.0: Evaluating language models on real-world enterprise text-to-sql workflows. In: ICLR

Lewis P, Perez E, Piktus A, et al (2020) Retrieval-augmented generation for knowledge-intensive nlp tasks. In: NeurIPS

Li J, Hui B, Qu G, et al (2023) Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. In: NeurIPS

Qwen Team (2024) Qwen2.5 technical report. arXiv preprint arXiv:241215115

Reimers N, Gurevych I (2019) Sentence-bert: Sentence embeddings using siamese bert-networks. In: EMNLP

Sun J, Xu C, Tang L, et al (2024) Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph. In: ICLR

Tang J, et al (2025a) Grapharena: Benchmarking large language models on graph computational problems. In: ICLR

Tang Z, Zhang Y, Cai S, et al (2025b) Llm-fk: Multi-agent llm reasoning for foreign key detection in large-scale complex databases. arXiv preprint arXiv:260307278

Team G, et al (2024) Gemini: A family of highly capable multimodal models. arXiv preprint arXiv:231211805

Transaction Processing Performance Council (2021) Tpc benchmark ds standard specification. Tech. rep., TPC

Wang H, Feng S, He T, et al (2023) Nlgraph: Can llms solve graph problems in natural language? In: NeurIPS

Yu T, Zhang R, Yang K, et al (2018) Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. In: EMNLP

Yuan Z, et al (2025) Gracore: Benchmarking graph comprehension and complex reasoning in large language models. In: COLING

Zhao Y, Zhao C, Nan L, et al (2023) Robut: A systematic study of table qa robustness against human-annotated adversarial perturbations. In: ACL

A Evaluation Details

A.1 Models

We evaluate three frontier-class LLMs:

- **Gemini 2.5 Flash** (Google, closed-weight, accessed via the Generative Language API). A multimodal model optimized for speed and accuracy at scale.
- **DeepSeek-V3** (DeepSeek, 671B parameter Mixture-of-Experts, MIT licensed, accessed via API). An open-weight model competitive with closed-source alternatives.
- **Qwen2.5-72B** (Qwen Team, 72B dense model, Apache-2.0, accessed via API). An open-weight model with strong reasoning capabilities.

All three models use greedy decoding (τ=0, p=1.0) with a single deterministic pass per question for exact reproducibility. No few-shot examples or chain-of-thought prompting is used.

A.2 Scoring Methodology

Exact Match (EM) is the primary metric. The predicted answer must exactly match the gold answer after the following canonicalization steps:

1. *List canonicalization*: lists are lowercased and sorted alphabetically; order does not matter.
2. *Path validation*: for `join_path` questions, alternative shortest paths of equal length are validated against the FK adjacency matrix to accept valid alternatives.
3. *Boolean normalization*: "yes"/"true"/"1" are equivalent; "no"/"false"/"0" are equivalent.
4. *Whitespace and case*: all comparisons are case-insensitive after stripping whitespace.

F1 is computed as token-level overlap between predicted and gold answer sets, used as a secondary metric for partial credit.

A.3 Baseline Implementation Details

Vector-RAG uses `all-MiniLM-L6-v2` Reimers and Gurevych (2019) embeddings (384 dimensions) with FAISS exact search (k=15 retrieved chunks per question). Schema elements (tables, columns, FK edges, lineage edges) are embedded as natural language descriptions.

Graph-Augmented computes 3-hop BFS neighborhoods from mentioned table names using NetworkX. The subgraph is serialized as structured text (node list + edge list with types).

Tool-Use provides 9 graph algorithm tools: `shortest_path`, `connected_components`, `check_fk_adjacency`, `get_lineage_forward`, `get_lineage_reverse`, `transitive_lineage`, `get_component_of`, `get_fk_neighbors`, `list_tables`. The LLM receives tool definitions and a schema summary, then generates up to 3 tool calls per question in a multi-turn conversation.

ReAct-Code provides a Python REPL with NetworkX pre-loaded. The LLM writes and executes Python code to query the schema graph directly, with up to

5 code execution rounds per question. This tests whether the LLM can translate graph-reasoning tasks into correct algorithmic implementations.

A.4 Statistical Treatment

All figures include 95% bootstrap confidence intervals computed via 2,000 resamples over questions. Per-question results (JSON files) are released for community bootstrap and stratification analyses. Greedy decoding ensures deterministic reproducibility without requiring multiple runs or confidence intervals over random seeds.

B Hop-Distance Analysis

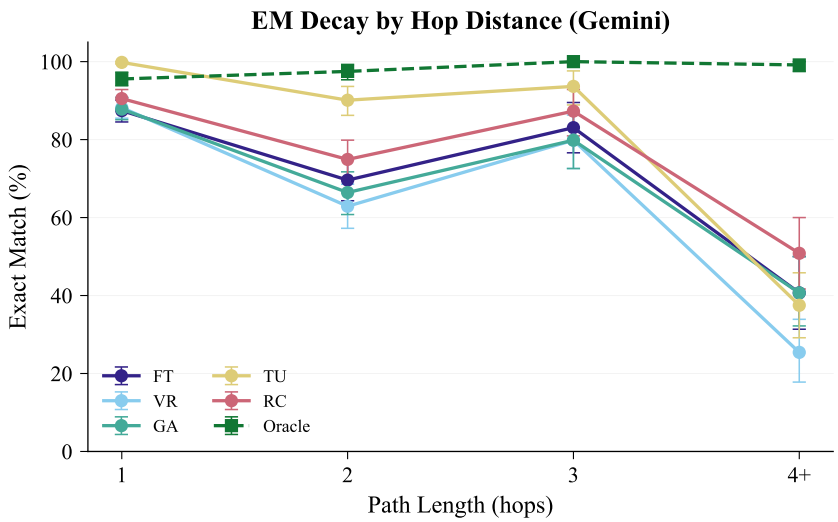

Fig. 8 EM by gold path length (Gemini). Static baselines collapse beyond 3 hops while Tool-Use degrades more gracefully, confirming that interactive graph access is essential for multi-hop reasoning.

Table 7 Minimum evidence hops per subtype (Syn-Logistics). 45.4% of questions exceed the GA 3-hop BFS radius, explaining the performance gap between static and agentic baselines.

Subtype	n	Exceeds 3h	%	GA EM
count	26	26	100%	100%
membership	20	20	100%	0%
combined_impact	34	34	100%	0%
full_enum	25	25	100%	100%
connected	25	25	100%	100%
isolation	25	25	100%	100%
transitive	29	4	14%	93%
hop_count	44	5	11%	100%
direct_fk	36	3	8%	100%
join_path	26	2	8%	81%
multi_source	42	0	0%	95%
forward	20	0	0%	100%
reverse	20	0	0%	100%
Total	372	169	**45.4%**	83.1%

C Dataset Construction Details

C.1 Syn-Logistics Design

The Syn-Logistics dataset was constructed to guarantee $n \geq 20$ for every subtype by building three parallel 4-hop lineage chains across five connected components. Table names are drawn from a Domain Dictionary of realistic supply-chain terminology to prevent semantic collapse under obfuscation. The schema contains 64 tables, 96 FK edges, 35 lineage edges, and 372 Tier 1 + 433 Tier 2 questions.

C.2 Lineage Edge Curation

Lineage (derived_from) edges are curated per dataset:

- **AdventureWorks**: Extracted from official Microsoft reference documentation mapping OLTP source tables to DW dimension/fact tables (39 edges).
- **TPC-DS**: No lineage edges; schema is a pure star/snowflake FK graph.
- **TPC-DI**: Derived from the official TPC-DI specification defining staging→dimension→fact transformation flows (21 edges).
- **OMOP CDM**: Derived from OHDSI ETL convention guides mapping source vocabularies and clinical tables to standardized CDM tables (21 edges).
- **Syn-Logistics**: Defined by construction: three parallel 4-hop chains (raw→staging→dimension→mart) per connected component (35 edges). All lineage edges are validated by verifying that the source table's schema semantically feeds the target table's schema.

D Example Questions and Predictions

Table 8 Representative questions across difficulty levels and categories, with gold answers and Tool-Use (TU) predictions. ✓ = correct, ✗ = incorrect. Tool-Use succeeds on lookup-style questions but fails on exhaustive enumeration and multi-hop composition.

Subtype	Question	Gold	TU Pred.
`direct_fk` *Easy*	Is there a direct FK between `DW.DimProduct` and `DW.FactInternetSales`?	yes	yes ✓
`join_path` *Medium*	What is the shortest FK path from `DW.DimCustomer` to `DW.DimProduct`?	[DimCustomer, FactInternetSales, DimProduct]	[DimCustomer, FactInternetSales, DimProduct] ✓
`membership` *Hard*	Which connected component contains `DW.DimCurrency`? List all tables in that component.	[DimCurrency, FactCurrencyRate, DimDate, ...] (12 tables)	[DimCurrency] ✗
`combined_impact` *Hard*	Which tables are transitively affected (via lineage and FK) if `raw_purchase_orders` is modified?	[stg_purchase_orders, dim_supplier, ...] (8 tables)	[stg_purchase_orders] ✗
`cascade_count` *Tier 2*	How many downstream rows would be affected if we delete `shipment_id=SH042` from `raw_shipments`?	147	147 ✓
`multi_hop_trace` *Tier 2*	Trace the full provenance chain of `order_id=ORD-2891` in `mart_profit_analysis`.	[raw_purchase_orders → stg_orders → dim_product → mart_profit_analysis]	[raw_purchase_orders, mart_profit_analysis] ✗

E Reproducibility Statement

All code, data, and evaluation scripts are publicly available at https://github.com/AJamal27891/dw-bench. The benchmark datasets (including schema graphs, QA pairs, and value-level data) will be released on HuggingFace at https://huggingface.co/datasets/AJamal27891/dw-bench. Per-question result JSON files enable exact reproduction of all reported numbers. Greedy decoding (τ=0) ensures deterministic outputs.